

VIETNAM NATIONAL UNIVERSITY OF HOCHIMINH CITY
THE INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



Data Mining
IT160IU
REPORT

Topic: Sentimental Analysis chatbot

By Group 1 – Member List

Number	Full Name	Student ID	Contribution
1	Dương Thị Quỳnh Như	ITCSIU22202	100%
2	Danny Donnelly	Udayton	Collaboration
3	Justin Butler	Udayton	Collaboration

ACKNOWLEDGE

We would like to extend our heartfelt gratitude to everyone who supported us in completing this report on the development of a Sentimental Analysis Chatbot.

Firstly, we are deeply thankful to our instructor, Nguyen Thi Thanh Sang, whose expertise, guidance, and encouragement have been invaluable insights and feedback that have greatly shaped the direction and quality of our work.

Secondly, we also want to express our appreciation to our team leader and all team members: Dương Thị Quỳnh Như, Justin Butler, and Danny Donnelly. Each member's hard work, dedication, and collaborative spirit played an essential role in making this project a success. The diverse skills and perspectives within our team enriched our discussions and enabled us to tackle challenges together effectively.

Thirdly, we would like to deeply appreciate to Muhammad Abdullah, the author of the Emotion dataset uploaded in Kaggle [1]. His working and sharing have provided the core resources for us to train and test our model and complete this report.

Finally, we are immensely grateful to our families and friends for their understanding and unwavering support throughout this journey. Their encouragement and belief in us provided the motivation we needed to overcome every challenge and complete this report.

In summary, thank you to everyone who contributed, in ways both large and small, to help us bring this project to life.

TABLE OF CONTENTS

ACKNOWLEDGE.....	2
TABLE OF CONTENTS.....	3
LIST OF FIGURES.....	4
ABSTRACT.....	5
CHAPTER 1: INTRODUCTION.....	6
I. Background.....	6
II. Problem statement.....	6
III. Scope and Objective.....	7
1. Scope.....	7
2. Objective.....	7
3. Structure of the report.....	7
CHAPTER 2: DATA PRE-PROCESSING.....	9
I. Data Overview.....	9
II. Data cleaning and preprocessing.....	11
III. Data analysis.....	16
1. Token length.....	16
2. Token frequency.....	19
3. Common token for each label.....	19
CHAPTER 3: CLASSIFICATION AND FIRST PREDICTION ALGORITHM	
IMPLEMENTATION.....	25
I. Logistic regression.....	25
II. Implementation.....	27
CHAPTER 4 :SECOND PREDICTION ALGORITHM IMPLEMENTATION.....	29
I. Improvement in tokenization.....	29
II. Validating best hyper parameter.....	30

III. Implementation.....	31
CHAPTER 5 : MODEL EVALUATION.....	33
I. Used formula definition.....	33
II. Logistic Regression pre modification.....	35
III. Logistic regression post modification.....	39
IV. Compare model with other models on the field.....	43
CHAPTER 6: CONCLUSION.....	44
I. Conclusion.....	44
II. Future Prospects.....	44
REFERENCES.....	45

LIST OF FIGURES

Picture 1: Emotion dataset from Kaggle.....	11
Picture 2: Dataset label counting.....	12
Picture 3: Bar chart of dataset label counting.....	13
Picture 4: Use re library to remove extra space.....	14
Picture 5: Use contractions library to remove contractions.....	14
Picture 6: Use neattext library to remove user handles in dataset.....	14
Picture 7: Use neattect library to remove stop words in dataset.....	15
Picture 8: Use nltk library to lemmatize input text.....	15
Picture 9: Use spellchecker library to check for miss spelled vocabulary.....	16
Table 1: Data after checking miss spelled word.....	16
Table 2: Meaningless data.....	17

Picture 10: Input text after cleaning.....	18
Picture 11: Histogram of token length.....	18
Picture 12: Mean, mode, median, standard deviation, max, min calculation.....	19
Picture 13: Box plot of Sequence token lengths.....	20
Picture 14: Line graph of token frequency of 30 most used token.....	21
Picture 15: Table show unique words for each label.....	21
Picture 16: Word cloud of most appeared words in all text.....	22
Picture 17: Word cloud of most appeared words in different labels.....	23
Table 4: Optimizer method of logistic regression.....	29
Picture 18: Tokenize input data.....	29
Picture 19: Logistic Regression with Stochastic gradient descent optimizer.....	30
Picture 20: Tokenize input text with modification.....	31
Picture 21: Validating best hyper parameter.....	32
Picture 22: Modification of logistic regression.....	33
Picture 23: Classification report of Stochastic descent optimizer for Logistic regression.....	37
Picture 24: Heat map of Stochastic descent optimizer for Logistic regression.....	38
Picture 25: Precision curve of Stochastic descent optimizer for Logistic regression.....	39
Picture 26: ROC curve of Stochastic descent optimizer for Logistic regression.....	39
Picture 27: Classification report of modified Stochastic descent optimizer for Logistic regression.....	41
Picture 28: Heat map of modified Stochastic descent optimizer for Logistic regression.....	42
Picture 29: Precision curve of modified Stochastic descent optimizer for Logistic regression.....	43
Picture 30: ROC curve of modified Stochastic descent optimizer for Logistic regression.....	43
Picture 31: Classification report of complementNB of 8000 features.....	46
Picture 32: Classification report of multinomialNB of 8000 features.....	47
Table 5: Loss function can be applied to GBM.....	49

Picture 33: Light Gradient Boosting Machine implementation.....	50
Picture 34: Classification report of Light Gradient Boosting Machine.....	50
Picture 35: Heat map of Light Gradient Boosting Machine.....	51
Figure 4.19: Add videos to lessons - 1.....	47
Figure 4.19: Add videos to lessons - 2.....	48
Figure 4.21: Preview course.....	48
Figure 4.22: Github Actions CI/CD pipeline - Build Job.....	49
Figure 4.23: Github Actions CI/CD pipeline - Deploy Job.....	49
Figure 5.1: Snapshot of the tabs from our workspace.....	62
Figure 5.2: A capture of the Project Tracking tab.....	62
Figure 5.3: A capture of Team tasks assignment.....	62
Figure 5.4: Level of Completion of Each team.....	63

LIST OF TABLES

Table 2.1: In-charge Table.....	13
Table 3.1: Functional requirements and Non-functional requirements.....	15
Table 4.1: Test case 1 for User Authentication & Authorization.....	50
Table 4.2: Test case 2 for User Authentication & Authorization.....	52
Table 4.3: Test case 1 for Course Management.....	54
Table 4.4: Test case 3 for Course Management.....	55
Table 4.5 Test case 4 for Course Management.....	56
Table 4.6: Test case 1 for Integration testing.....	60

ABSTRACT

This report outlines the development process of **Sentimental Analysis chatbot**, a chatbot utilizing Natural Language Processing to extract users' emotions conveyed in their sentence input. The system addresses challenges commonly faced in nowadays business, offering a better chance to understand customers behavior and insights from which can increase user experience and build business reputation. Built using well known data mining algorithms like K nearest neighbour and Logistic regression with Python library support and Emotion Dataset from Kaggle [1], **Sentimental Analysis chatbot** enhances our understanding of the deep meaning and feeling of users through their sentences and stories.

CHAPTER 1

INTRODUCTION

I. Background

Sentimental analysis, or opinion mining, is a process of analyzing digital text to determine whether emotions it conveys [11]. It is a domain of natural processing language, using software to understand insights and emotion. It focuses not only on popularity (positive, negative, neutral) but also on emotions (sad, happy, surprise, etc).

The rise of Artificial Intelligence recently has redefined the way businesses analyze and interpret opinions across online platforms [15]. Sentimental analysis has become a popular tool for businesses to decode opinions shared via online platforms about their products and provide insights into customer feelings, preferences, and needs. Sentimental analysis is a popular domain in today's research since it plays a vital role for businesses to attract their customers and give competitive edge compared to other companies in the same field. This method supports nowadays companies to extract meaningful insight from customers that lead to a better decision, deliver a strong customer experience and improve brand reputation [12]. Moreover, it also allows companies to evaluate the effectiveness of their marketing campaign via online discussion and social media mentions with positive response means the campaign has successfully delivered to their target audience while negative response illustrates that the project needs adjustment.

II. Problem statement

Although there are sentimental analysis chatbots which have provided numerous advantages, there still exist several problems that need to be addressed to ensure a more robust and effective emotional detection model.

With the rapid pace of global digitalization, it is necessary for every business to apply new technology to increase their performance and income effectively. Driven by this belief, we set out to create Sentimental Analysis chatbot, a natural language processing model focused on analyzing user input and extract meaningful emotion conveyed in it effectively and efficiently.

One of the major concerns is the accuracy of the model. As far as we have researched, some models recently being used have given very low accuracy with the same dataset which can lead to miss classification of the labels of emotion.

Another concern is the time for processing input. Some algorithms used currently may give high accuracy; however, the time for processing the input is extremely high due to the high complexity of the algorithm which can lead to low performance when being used in real life for detecting real time emotions.

III. Scope and Objective

1. Scope

The purpose of this project is to develop and evaluate an emotional detection chatbot that can both give high accuracy and low performance time. It can address the aforementioned challenges while preprocessing data effectively.

2. Objective

By leveraging data mining best practices, the system aims to achieve the following goals:

- Equip models with the most suitable algorithms with the best hyper parameters.
- Handle the low time needed to process data input.

3. Structure of the report

Acknowledgements: A section to express gratitude for support received during the project.

Abstract: A summary of the project's purpose, methodology, and outcomes.

Chapter I - Introduction: Covers the background, team motivation, project scope, implementation overview, and report structure,

Chapter II - Data preprocessing and data analysis: Explain the preprocessing and cleaning data steps and analysis of data after processing

Chapter III - Classification and first prediction algorithm implementation: Explain the first algorithm and the approach utilizing and implementing the first algorithm.

Chapter IV - Second prediction algorithm implementation: Describes the implementation of the second algorithm and the approach utilizing and implementing the second algorithm.

Chapter V - Model Evaluation: Evaluates the results, compares progress with initial plans, and discusses insights gained.

Chapter VI - Conclusion: Summarizes key findings, shortcomings, and future prospects of the project.

CHAPTER 2

DATA PRE-PROCESSING

I. Data Overview

The Emotion dataset from Kaggle is a collection of Twitter messages meticulously annotated with six fundamental emotions: anger, fear, joy, love, sadness, and surprise which is helpful for sentiment analysis, emotion classification and textual analysis [1].

This dataset includes 417000 different data with no missing or unmatched data.

This dataset has csv file type and consists of 3 main columns: id column, label column and text input column (as shown in picture 1).

A	B	C
	text	label
0	i just feel really helpless and heavy hearted	4
1	ive enjoyed being able to slouch about relax and unwind and frankly needed it after those last few weeks around the end of uni and the ex	0
2	i gave up my internship with the dmrg and am feeling distraught	4
3	i dont know i feel so lost	0
4	i am a kindergarten teacher and i am thoroughly weary of my job after having taken the university entrance exam i suffered from anxiety fo	4
5	i was beginning to feel quite disheartened	0
6	i would think that whomever would be lucky enough to stay in this suite must feel like it is the most romantic place on earth	2
7	i fear that they won t ever feel that delicious excitement of christmas eve at least not in the same way i remember doing it	1
8	im forever taking some time out to have a lie down because i feel weird	5
9	i can still lose the weight without feeling deprived	0
10	i try to be nice though so if you get a bitchy person on the phone or at the window feel free to have a little fit and throw your pen at her face	1
11	im feeling a little like a damaged tree and that my roots are a little out of wack	0
12	i have officially graduated im not feeling as ecstatic as i thought i would	1
13	i feel like a jerk because the library students who all claim to love scrabble cant be bothered to participate and clearly scrabble is an inap	3
14	i feel my portfolio demonstrates how eager i am to learn but some who know me better might call it annoyingly persistent	1
15	i may be more biased than the next because i have a dependent life to take care of and to keep safe but i feel we all need to take care of ou	1
16	i didn t feel terrific	1
17	i miss all the others as well that feel that i wronged them and they will soon understand that i didnt	3
18	i feel so stupid that i realise it so late	0
19	i saunter through the airport terminals feeling that i have had an experience that renders the petty tribulations of everyday travel someho	3
20	i need to feel dangerous and pretty so here a striking dance pick deep in vogue minutes ago	3
21	i am feeling much stronger and more confident now and by professional opinion i know that i do not have anything serious	1
22	i take a shower i feel wonderful energetic and all my previous feelings about my life turn into this awesome feeling creating my life like the	1
23	i don t feel submissive and for the time being i ve lost interest in some bdsm stuff	0
24	i would imagine this is just one of the reasons why marriage is so hard because theyll see all the good bad and ugly parts of you the parts	0
25	i feel like a real fan not that i was ever a fake fan	0

Picture 1: Emotion dataset from Kaggle

The text input of the second column is a nominal data which includes a free-form text string while the label column is also a nominal data that have 6 different categories (sadness (0), joy (1), love (2), anger (3), fear (4), and surprise (5)).

An example of text input of 6 categories:

- Label 0 (sadness): “ive enjoyed being able to slouch about relax and unwind and frankly needed it after those last few weeks around the end of uni and the expo i have lately started to find myself feeling a bit listless which is never really a good thing”
- Label 1 (joy): “i have officially graduated im not feeling as ecstatic as i thought i would”
- Label 2 (love): “i would think that whomever would be lucky enough to stay in this suite must feel like it is the most romantic place on earth”
- Label 3 (anger): “i miss all the others as well that feel that i wronged them and they will soon understand that i didnt”
- Label 4 (fear): “i just feel really helpless and heavy hearted”
- Label 5 (surprise): “im feeling a bit amazed and grateful about having landed amid such a congregation”

✓
0
giây

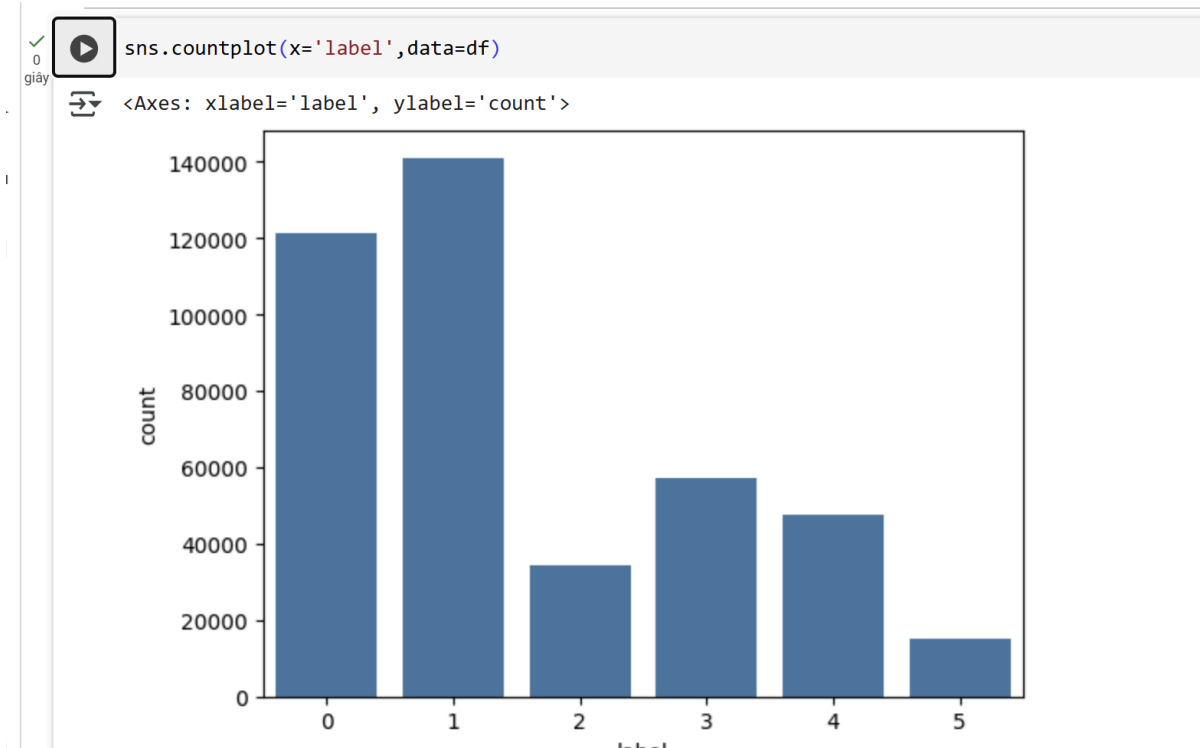
```
df['label'].value_counts()
```



	count
label	
1	141067
0	121187
3	57317
4	47712
2	34554
5	14972

dtype: int64

Picture 2: Dataset label counting



Picture 3: Bar chart of dataset label counting

In picture 2, by using seaborn library from Python, there are approximately 120000 data of label 0 (sadness), 140000 data of label 1 (joy), 34000 data of label 2 (love), 57000 data of label 3 (anger), 88000 of label 4 (fear) and 15000 of label 5 (surprise).

II. Data cleaning and preprocessing

Since the text input is a free-form text string, our preprocessing steps include: cleaning, tokenization lemmatization, stop-word removal, and part-of-speech tagging [2].

In the cleaning phase, we implement neattext package which is known for its ability of cleaning text data for Natural Language Processing [3].

- Firstly, extra spaces are eliminated from text because different amounts of spaces between words can cause the model to see them as different inputs. Hence, the data has less noise, and more meaningful features. Moreover, many NLP models (e.g., using tokenizers like nltk, spaCy, transformers) split text based on spaces so that extra spaces

can lead to confusion and create empty or incorrect tokens. In this stage, re library which is known for its ability in pattern matching and text manipulation to remove extra space [4].

```
import re

# Function to remove multiple spaces
def remove_multiple_spaces(text):
    return re.sub(r'\s+', ' ', text).strip()
df['Clean_Text'] = df['text'].apply(remove_multiple_spaces)
```

Picture 4: Use re library to remove extra space

- Secondly, we expand contractions from text input. Since the data is extracted from Twitter blog, it contains a variety of contractions which can lead to inconsistent and unclean input. For example, “Don’t” and “Do not” sound identical for people; however, from a model perspective, it is totally different and can be transformed into two different kinds of tokens. In this phase, the contractions library which is known for its ability to expand contraction in English text [5].

```
[45] import contractions
df['Clean_Text'] = df['text'].apply(lambda x: contractions.fix(x))
df['Clean_Text'] = df['Clean_Text'].apply(lambda x: x.replace(' ill ', ' I will '))
```

Picture 5: Use contractions library to remove contractions

- Thirdly, we remove usernames from text since the dataset is collected from social media (Twitter) which includes a variety of mentions of specific users that are not useful for our machine learning task.

```
# User handles
df['Clean_Text'] = df['text'].apply(nfx.remove_userhandles)
```

Picture 6: Use neattext library to remove user handles in dataset

- Fourthly, we remove stop words which are common words that do not carry much meaning in Natural Language Processing tasks such as: the, a, an, etc which will reduce noise in the dataset.

```
[ ] # Stopwords
df['Clean_Text'] = df['Clean_Text'].apply(nfx.remove_stopwords)
```

Picture 7: Use neattect library to remove stop words in dataset

- Fifthly, we lemmatize the data input to reduce a word to its base form to reduce variants and improve model accuracy by reducing sparsity in data by applying Natural Language Toolkit (nltk) library. For example, transform running to run, better to good.

```
import nltk
from nltk.stem import WordNetLemmatizer
from nltk.corpus import wordnet
nltk.download('punkt')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')

lemmatizer = WordNetLemmatizer()
from nltk.corpus import wordnet
def get_wordnet_pos(word):
    tag = nltk.pos_tag([word])[0][1][0].upper()
    tag_dict = {"J": wordnet.ADJ,
                "N": wordnet.NOUN,
                "V": wordnet.VERB,
                "R": wordnet.ADV}
    return tag_dict.get(tag, wordnet.NOUN)
def lemmatize_text(text):
    words = nltk.word_tokenize(text)
    lemmatized_words = [lemmatizer.lemmatize(word, get_wordnet_pos(word)) for word in words]
    return ' '.join(lemmatized_words)

df['Clean_Text'] = df['Clean_Text'].apply(lemmatize_text)
```

Picture 8: Use nltk library to lemmatize input text

- Sixthly, since it is Twitter blog which contains a variety of miss spelled words that create noise for the model, we apply spellchecker library to transform those miss spelled word into a correct version which has been illustrated in picture 9. After this step, all wrongly written vocabulary has been corrected as shown in table 1.

```

import re
import pandas as pd
from spellchecker import SpellChecker

spell = SpellChecker()
def reduce_elongation(word):
    return re.sub(r'(\.){1{2,}}', r'\1', word)
def clean_word(word):
    word = reduce_elongation(word.lower())
    if word in spell:
        return word
    correction = spell.correction(word)
    if not correction or correction not in spell:
        return ''

    return correction
def clean_text(text):
    words = text.split()
    cleaned_words = [clean_word(w) for w in words]
    return ' '.join(filter(None, cleaned_words))

df['Final_Text'] = df['Clean_Text'].apply(clean_text)

```

Picture 9: Use spellchecker library to check for miss spelled vocabulary

id	After lemmatization in step 5	After checking miss spelled in step 6
2933	love cuddle lot affectionate little silly nice sweet thing care get paranoid feel soooooooooooooo giggly	love cuddle lot affectionate little silly nice sweet thing care get paranoid feel so giggly
165864	reachad home find baby bath fed	reached home find baby bath fed

Table 1: Data after checking miss spelled word

- Seventhly, after processing, sentences that are meaningless such as the one shown in table 2 will be dropped.

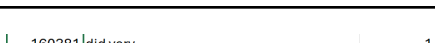
Index	Text	Label	In kaggle
125450	once	4	
6689	in	0	
46216	sometimes	0	
160381	did very	1	

Table 2: Meaningless data

df.head(20)

by

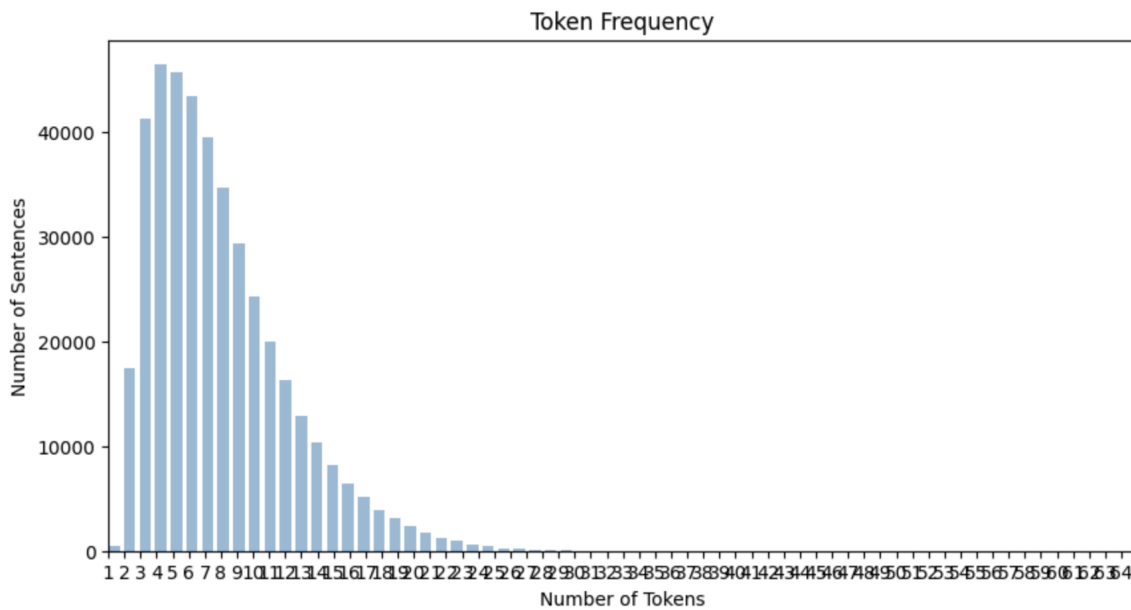
	Unnamed: 0	text	label	Clean_Text
0	0	i just feel really helpless and heavy hearted	4	[feel, helpless, heavy, hearted]
1	1	ive enjoyed being able to slouch about relax a...	0	[ive, enjoyed, able, slouch, relax, unwind, fr...
2	2	i gave up my internship with the dmrg and am f...	4	[gave, internship, dmrg, feeling, distraught]
3	3	i dont know i feel so lost	0	[dont, know, feel, lost]
4	4	i am a kindergarten teacher and i am thoroughl...	4	[kindergarten, teacher, thoroughly, weary, job...
5	5	i was beginning to feel quite disheartened	0	[beginning, feel, disheartened]
6	6	i would think that whomever would be lucky eno...	2	[think, whomever, lucky, stay, suite, feel, li...
7	7	i fear that they won t ever feel that deliciou...	1	[fear, feel, delicious, excitement, christmas,...
8	8	im forever taking some time out to have a lie ...	5	[im, forever, taking, time, lie, feel, weird]
9	9	i can still lose the weight without feeling de...	0	[lose, weight, feeling, deprived]
10	10	i try to be nice though so if you get a bitchy...	1	[try, nice, bitchy, person, phone, window, fee...
11	11	im feeling a little like a damaged tree and th...	0	[im, feeling, little, like, damaged, tree, roo...
12	12	i have officially graduated im not feeling as ...	1	[officially, graduated, im, feeling, ecstatic,...
13	13	i feel like a jerk because the library student...	3	[feel, like, jerk, library, student, claim, lo...
14	14	i feel my portfolio demonstrates how eager i a...	1	[feel, portfolio, demonstrates, eager, learn, ...
15	15	i may be more biased than the next because i h...	1	[biased, dependent, life, care, safe, feel, ne...

Picture 10: Input text after cleaning

In conclusion, after preprocessing, as shown in picture 10 in the Clean_Text column, the string input is transformed into an array of meaningful words and 416781 sentences remained compared to 416809 sentences from the beginning.

III. Data analysis

1. Token length



Picture 11: Histogram of token length

```

import pandas as pd
import numpy as np
from scipy import stats
token_lengths = df['Final_Text'].apply(lambda x: len(x.split()))
length_series = pd.Series(token_lengths)
mean_length = length_series.mean()
median_length = length_series.median()
mode_length = length_series.mode().iloc[0]
std_dev = length_series.std()
min_length = length_series.min()
max_length = length_series.max()
print(f"Mean Length: {mean_length:.2f}")
print(f"Median Length: {median_length}")
print(f"Mode Length: {mode_length}")
print(f"Standard Deviation: {std_dev:.2f}")
print(f"Min Length: {min_length}")
print(f"Max Length: {max_length}")

```

```

Mean Length: 7.79
Median Length: 7.0
Mode Length: 4
Standard Deviation: 4.37
Min Length: 1
Max Length: 65

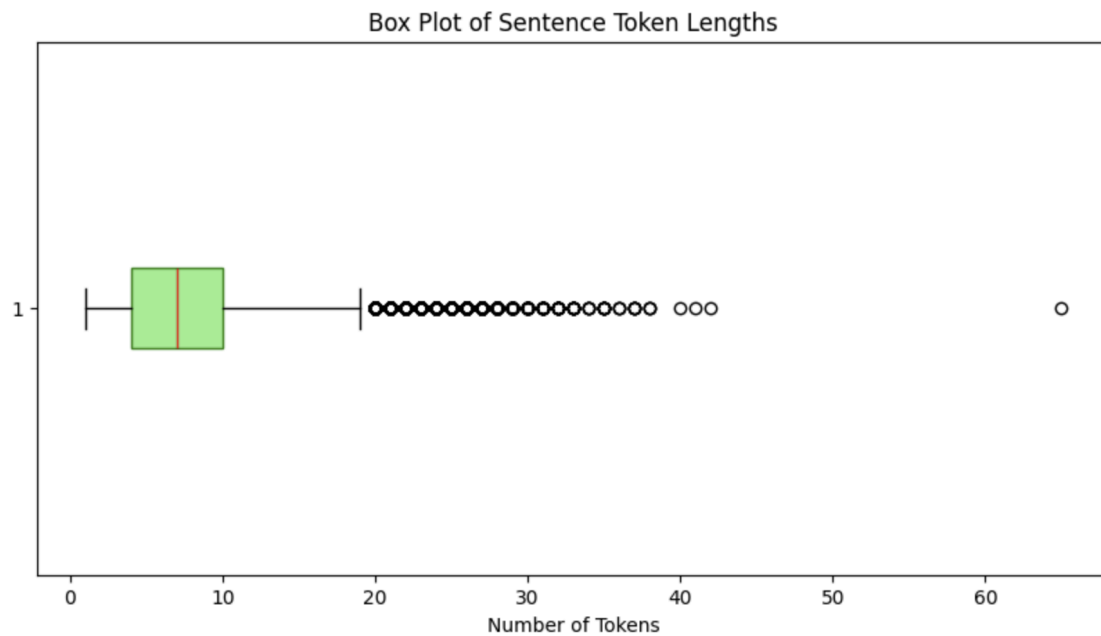
```

Picture 12: Mean, mode, median, standard deviation, max, min calculation

Mean	Median	Mode	Standard deviation	Max	Min
7.79	7.0	4	4.37	65	1

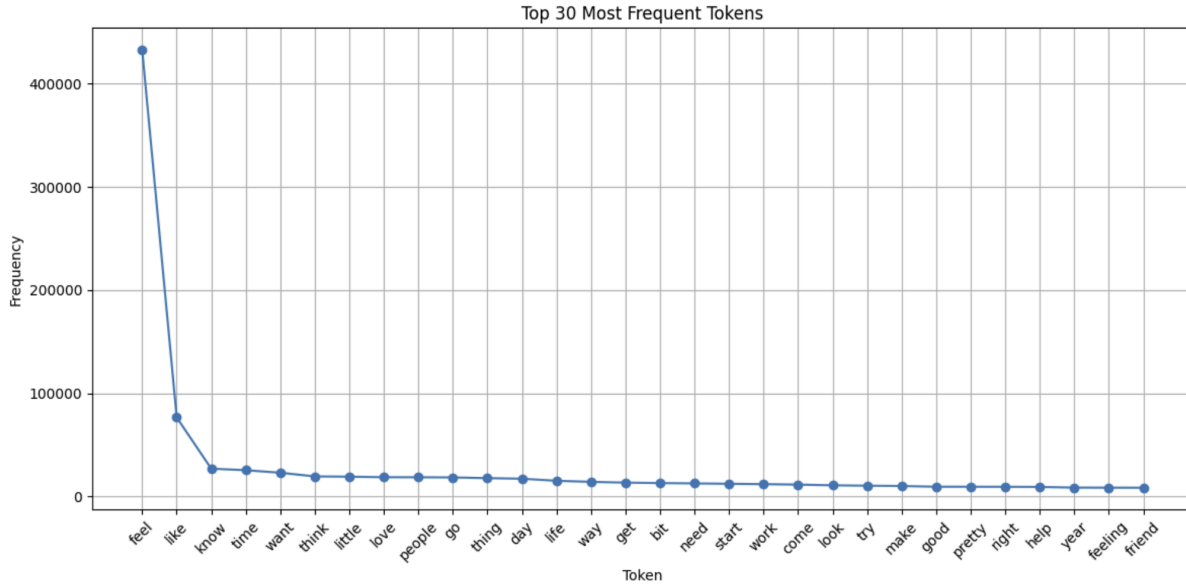
Table 3: Mean, median, mode, standard deviation, max and min

According to the histogram shown in picture 11 and calculated summarized numbers in Table 3, most of the sentences after being preprocessed have an average length of 8 tokens which indicates that the majority of text samples are relatively short and possibly single-sentence inputs or brief phrases. However, since the standard deviation of the dataset's sentence length is 4.53 it emphasizes that there is a lot of variation in sentence length with the shortest length being 0 and the most extended length being 66 with outlier lengths that are over 20, as shown in picture 10.



Picture 13: Box plot of Sequence token lengths

2. Token frequency



Picture 14: Line graph of token frequency of 30 most used token

According to picture 14, the most common token in the whole dataset is “feel” with being used approximately over 400,000 times and “like” with less than 100,000 times being used.

3. Common token for each label

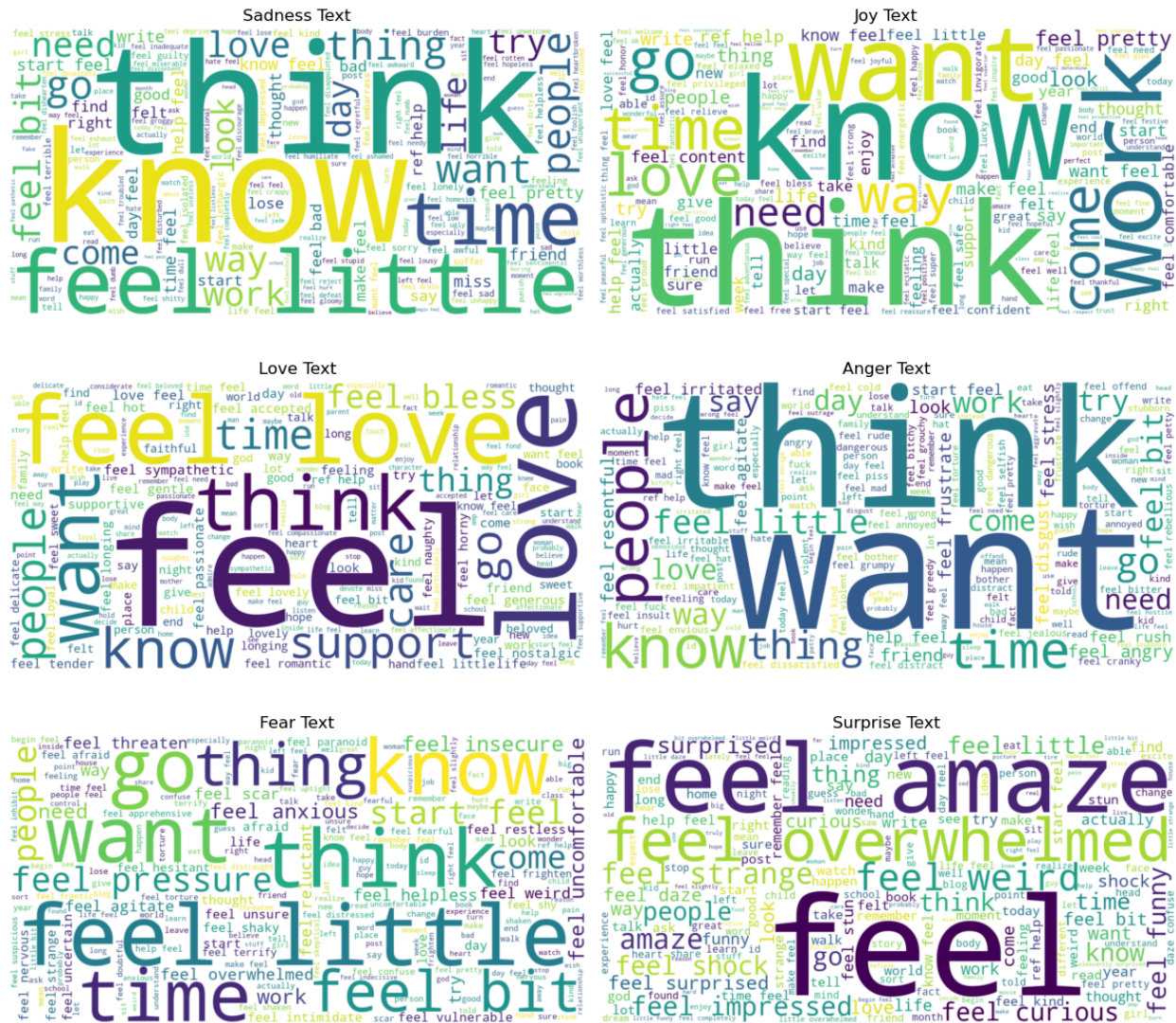
Category		Unique in Category (not in All)
0	Sadness	abuse, ache, awful, awkward, boring, burden, completely, defeat, depressed, disappointed, discourage, drain, dull, embarrass, emotional, exhaust, guilty, hat, helpless, homesick, hopeless, horrible, ignore, isolated, lethargic, lonely, low, miserable, numb, reject, shitty, sorry, stupid, suffer, terrible, unhappy, useless, worthless
1	Joy	accepted, bless, calm, comfortable, confident, content, cool, creative, cute, enjoy, excite, free, glad, important, inspire, lucky, ok, perfect, positive, productive, proud, relaxed, respect, safe, satisfied, special, strong, successful, super, thankful, trust, value, wonderful
2	Love	accepted, affectionate, beloved, bless, compassionate, delicate, devote, face, faithful, fond, generous, gentle, hand, horny, hot, longing, lovely, loyal, naughty, nostalgic, old, passionate, romantic, slutty, supportive, sympathetic, tender, touch, understand, woman
3	Anger	agitate, annoyed, bitchy, bitter, bother, cold, cranky, dangerous, disgust, dissatisfied, distract, envious, frustrate, fuck, greedy, grouchy, grumpy, hat, heartless, hostile, impatient, insult, irritable, irritated, jealous, mad, offend, petty, piss, reason, rebellious, resentful, rude, rush, selfish, stubborn, torture, violent, wrong
4	Fear	afraid, agitate, anxious, apprehensive, confuse, distraught, distressed, doubtful, fear, fearful, frantic, frighten, helpless, hesitant, insecure, intimidate, nervous, overwhelmed, paranoid, pressure, reluctant, restless, scar, shaken, shaky, shy, skeptical, strange, suspicious, terrify, threaten, torture, uncertain, uncomfortable, unsure, uptight, vulnerable, weird
5	Surprise	body, completely, confuse, curious, daze, enthal, face, funny, head, impressed, learn, month, overwhelmed, see, sense, shock, sit, slightly, sort, story, strange, stun, surprised, weird

Picture 15: Table show unique words for each label

All Text



Picture 16: World cloud of most appeared words in all text



The picture 15, 16 and 17 above shows the words that are among the top 100 in each emotion category but do not appear in the top 100 for “All” tweets. Here's an analysis of the results:

- Sadness

Unique Words: “abuse, ache, awful, awkward, boring, burden, completely, defeat, depressed, disappointed, discourage, drain, dull, embarrass, emotional, exhaust, guilty, hat, helpless, homesick, hopeless, horrible, ignore, isolated, lethargic, lonely, low, miserable, numb, reject, shitty, sorry, stupid, suffer, terrible, unhappy, useless, worthless”

Analysis:

The uniqueness of the sadness category is wide with negative words like “abuse” “miserable”, which are emotionally heavy and expected in sad expressions. They are strong indicators of sadness and differentiates sadness from the other labels.

- Joy

Unique Words: “accepted, bless, calm, comfortable, confident, content, cool, creative, cute, enjoy, excite, free, glad, important, inspire, lucky, ok, perfect, positive, productive, proud, relaxed, respect, safe, satisfied, special, strong, successful, super, thankful, trust, value, wonderful”

Analysis:

The joy category includes clearly positive words like “successful” and “accepted” which are expected in joyful expressions. However, the presence of words like “important”, “ok”, “respect” (which are more generic) suggests that the uniqueness here is modest. These words might be used in contexts of joy but are not as strongly emotion-specific as those in other categories.

- Love

Unique Words: “accepted, affectionate, beloved, bless, compassionate, delicate, devote, face, faithful, fond, generous, gentle, hand, horny, hot, longing, lovely, loyal, naughty, nostalgic, old, passionate, romantic, slutty, supportive, sympathetic, tender, touch, understand, woman”

Analysis:

The love category exhibits a rich vocabulary that clearly relates to affection and romance. Words like "beloved", "compassionate", “accepted” and "bless" are highly specific to expressions of love. This strong set of unique words helps differentiate love from the overall language of tweets. However, there are words that are modest and also easy to be confused among others such as “naughty” or “old” and “women”.

- Anger

Unique Words: “agitate, annoyed, bitchy, bitter, bother, cold, cranky, dangerous, disgust, dissatisfied, distract, envious, frustrate, fuck, greedy, grouchy, grumpy, hat, heartless, hostile,

impatient, insult, irritable, irritated, jealous, mad, offend, petty, piss, reason, rebellious, resentful, rude, rush, selfish, stubborn, torture, violent, wrong”

Analysis:

The anger category shows a set of words that are directly linked to expressions of irritation and hostility. Words like "disgust," "annoyed," and "bitchy" are unambiguous markers of anger. The relatively short list indicates that these distinctive anger-related words are not as prevalent in the overall collection of tweets.

- Fear

Unique Words: “afraid, agitate, anxious, apprehensive, confuse, distraught, distressed, doubtful, fear, fearful, frantic, frighten, helpless, hesitant, insecure, intimidate, nervous, overwhelmed, paranoid, pressure, reluctant, restless, scar, shaken, shaky, shy, skeptical, strange, suspicious, terrify, threaten, torture, uncertain, uncomfortable, unsure, uptight, vulnerable, weird”

Analysis:

Fear is characterized by a variety of adjectives that capture apprehension and unease. Words such as “afraid”, “anxious” “nervous” and “terrify” clearly signal fear. However, the inclusion of more ambiguous terms like “unsure” and "weird" (which also appear in surprise) suggests there can be some overlap in how these emotions are linguistically expressed.

- Surprise

Unique Words: “body, completely, confuse, curious, daze, enthrall, face, funny, head, impressed, learn, month, overwhelmed, see, sense, shock, sit, slightly, sort, story, strange, stun, surprised, weird”

Analysis:

For surprise, many words indicate astonishment or a sudden reaction, such as "amazed," "shocked," "stunned," and "surprised." Interestingly, words like "overwhelmed," "strange," and "weird" also appear here and in fear, suggesting that these two emotions may share some linguistic cues in certain contexts. As shown above, the surprise vocabulary is not so distinctive compared to others which can lead to the low performance in this class.

In summary, the analysis of the unique words reveals that while some emotion categories (like love and anger) are expressed with a rich set of distinctive terms, others (like sadness and surprise) share much of their vocabulary with the overall tweet corpus. Understanding these nuances is crucial for refining both data preprocessing and model design in emotion classification tasks.

CHAPTER 3:

CLASSIFICATION AND FIRST PREDICTION ALGORITHM IMPLEMENTATION

I. Logistic regression

Logistic regression is supervised machine learning known for its classifying ability by estimating the probability of instances belonging to given classes by sigmoid function and choosing the label with highest probability[9].

Logistic regression is usually used for binary classification by applying function

$$P(y = 1|x) = \sigma(W^T x + b)$$

where $\sigma(z) = \frac{1}{1 + e^{-z}}$ is a Sigmoid function, W is the weight and b is bias

that measures the probability of input instances belonging to class 1 or class 0. However, for multi class problem such as our problem with 6 classes, Softmax algorithm

$$P(y = k|x) = \frac{e^{w_k^T x}}{\sum_j e^{w_j^T x}}$$

would be applied to measure the probability distribution over the whole class.

For logistic regression algorithm, in order to increase the prediction accuracy, cross-entropy or loss function is applied to measure the difference between the actual and predicted data point to have better weight W and bias b . As shown below are loss function for binary class classification and for multi class classification

$$\text{Loss function for binary class} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

where $\hat{y}$ is the predicted probability of class 1.

$$\text{Loss function for multi class} = - \sum_k^K y_k \log(\hat{y}_k)$$

where $\hat{y}_k$ is the predicted probability of class k.

After retrieving loss from loss function, gradient descent is computed by backpropagation to measure the direction and rate at which loss is increased. After that, one of optimization algorithms is applied to minimize loss by modifying weight and bias as shown in table 7.

Optimization method	Formula
Gradient descent	$w = w - \alpha \nabla_w \text{Loss}, b = b - \alpha \nabla_b \text{Loss}$
Stochastic gradient descent	$w = w - \alpha \nabla_w \text{Loss}(x_i, y_i)$
Momentum	$v_t = \beta v_{t-1} + \alpha \nabla_w \text{Loss}$ where β is momentum factor $w = w - v_t$
RMSprop	$E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta) g_t^2$
Adam (Adaptive moment estimation)	$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$ $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ Bias correct: $\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$ Weight update: $w = w - \frac{\alpha}{\sqrt{\hat{v}_t + \epsilon}} \times \hat{m}_t$

--	--

Table 4: Optimizer method of logistic regression

II. Implementation

Firstly, to tokenize the data input, we would like to utilize the Term Frequency - Inverse Document Frequency (TF-IDF) Vectorization to transform text into numbers with formula

$$TF - IDF(t, d) = TF(t, d) * DF(t, d) \text{ with}$$

$TF(t, d) = \frac{\text{Number of times term } t \text{ appear in document } d}{\text{Total of term in document } d}$ shows the importance of word in a single document and

$IDF(t) = \log\left(\frac{\text{total number of document}}{\text{number of document contain term } d}\right)$ illustrates the importance of the word across all documents and penalizes common words.

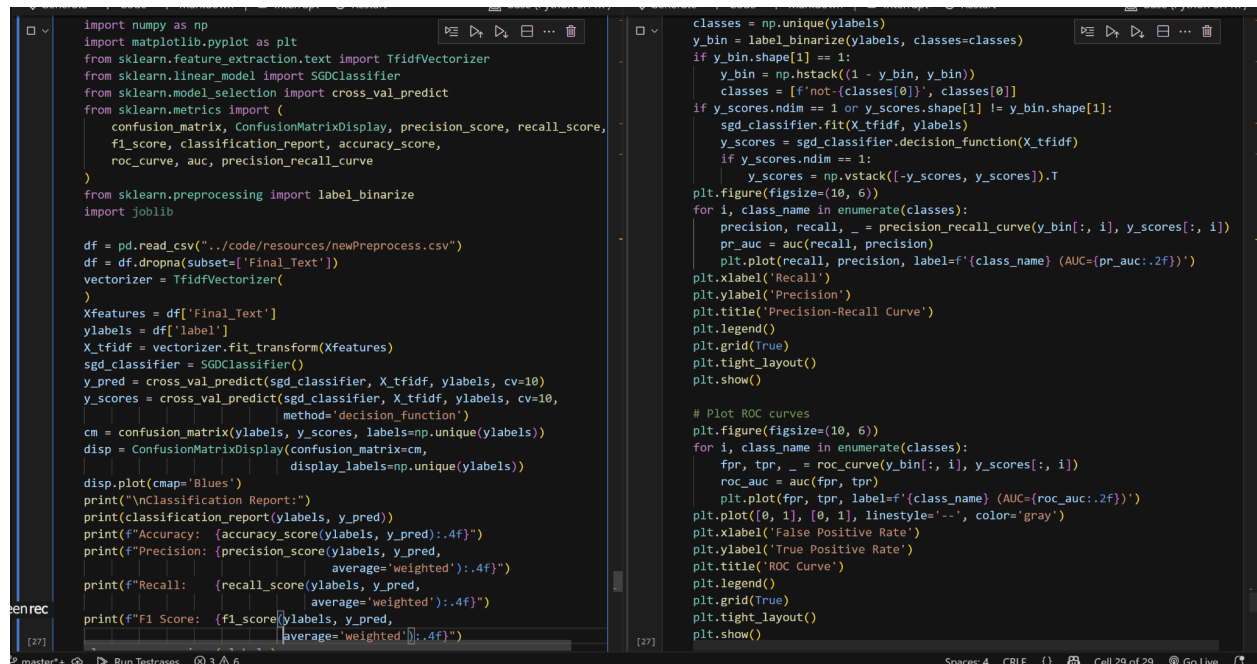
On default, the words are extracted on a unigram unit which means sentences are divided into multiple or single words as shown in picture 18. Moreover, we use the Euclidean formula to normalize the document vector to have norm of 1, which means the direction is kept but the magnitude is turned into 1.

```
vectorizer = TfidfVectorizer(
)
Xfeatures = df['Final_Text']
ylabels = df['label']
X_tfidf = vectorizer.fit_transform(Xfeatures)
```

Picture 18: Tokenize input data

For the implementation of logistic regression, we would like to apply the Stochastic gradient descent optimizer algorithm. Since our dataset is a large one with approximately 400000 rows of data, the original Gradient descent is not suitable since it computes Gradient using the entire dataset which slows down the training phase. Hence, we decide to test with Stochastic gradient descent, a variety of Gradient descent for big data, which is also supported by sklearn library of

Python. This algorithm can handle well with big data since it updates weights using a small portion of data samples.



```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import SGDClassifier
from sklearn.model_selection import cross_val_predict
from sklearn.metrics import (
    confusion_matrix, ConfusionMatrixDisplay, precision_score, recall_score,
    f1_score, classification_report, accuracy_score,
    roc_curve, auc, precision_recall_curve
)
from sklearn.preprocessing import label_binarize
import joblib

df = pd.read_csv("../code/resources/newPreprocess.csv")
df = df.dropna(subset=['Final_Text'])
vectorizer = TfidfVectorizer()
Xfeatures = df['Final_Text']
ylabels = df['label']
X_tfidf = vectorizer.fit_transform(Xfeatures)
sgd_classifier = SGDClassifier()
y_pred = cross_val_predict(sgd_classifier, X_tfidf, ylabels, cv=10)
y_scores = cross_val_predict(sgd_classifier, X_tfidf, ylabels, cv=10,
                             method='decision_function')
cm = confusion_matrix(ylabels, y_scores, labels=np.unique(ylabels))
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=np.unique(ylabels))
disp.plot(cmap='Blues')
print("\nClassification Report:")
print(classification_report(ylabels, y_pred))
print(f"Accuracy: {accuracy_score(ylabels, y_pred):.4f}")
print(f"Precision: {precision_score(ylabels, y_pred,
                                  average='weighted'):.4f}")
print(f"Recall: {recall_score(ylabels, y_pred,
                             average='weighted'):.4f}")
print(f"F1 Score: {f1_score(ylabels, y_pred,
                             average='weighted'):.4f}")

classes = np.unique(ylabels)
y_bin = label_binarize(ylabels, classes=classes)
if y_bin.shape[1] == 1:
    y_bin = np.hstack((1 - y_bin, y_bin))
    classes = [f'not-{classes[0]}', classes[0]]
if y_scores.ndim == 1 or y_scores.shape[1] != y_bin.shape[1]:
    sgd_classifier.fit(X_tfidf, ylabels)
    y_scores = sgd_classifier.decision_function(X_tfidf)
    if y_scores.ndim == 1:
        y_scores = np.vstack([-y_scores, y_scores]).T
plt.figure(figsize=(10, 6))
for i, class_name in enumerate(classes):
    precision, recall, _ = precision_recall_curve(y_bin[:, i], y_scores[:, i])
    pr_auc = auc(recall, precision)
    plt.plot(recall, precision, label=f'{class_name} (AUC={pr_auc:.2f})')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# Plot ROC curves
plt.figure(figsize=(10, 6))
for i, class_name in enumerate(classes):
    fpr, tpr, _ = roc_curve(y_bin[:, i], y_scores[:, i])
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, label=f'{class_name} (AUC={roc_auc:.2f})')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```

Picture 19: Logistic Regression with Stochastic gradient descent optimizer

CHAPTER 4 :

SECOND PREDICTION ALGORITHM IMPLEMENTATION

I. Improvement in tokenization

In order to improve the performance of the previous Logistic regression, I would like to apply a different tokenization method which is shown below in picture 15. As shown in picture 18, we applied the sublinear scaling to reduce the impact of high-frequency words and improve the sensitivity towards words that appear less frequently by using logarithm function, as shown in the formula below.

$$TF(t, d) = 1 + \log(TF_{raw}(t, d))$$

Moreover, not only unigram (single words) but also bigrams (pairs of words) are extracted to avoid missing two words supporting each other such as “brown fox”, etc. Moreover, we also ignore too common words since they do not have a high effect on distinguishing between labels and also remove less-appeared words to reduce noise in our dataset. Finally, the top 50000 features are kept based on their TF-IDF score to reduce dimensionality and improve efficiency.

```
from sklearn.feature_extraction.text import TfidfVectorizer

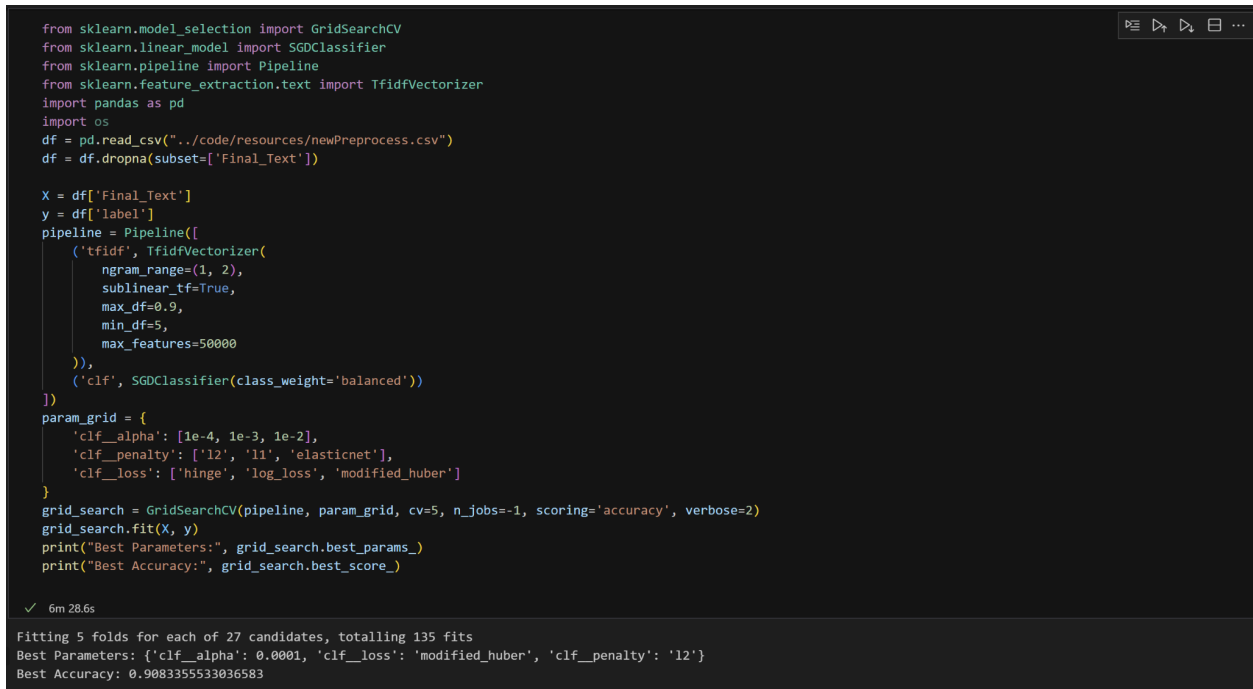
vectorizer = TfidfVectorizer(
    ngram_range=(1, 2),
    sublinear_tf=True,
    max_df=0.9,
    min_df=5,
    max_features=50000,
)

Xfeatures = df['Final_Text']
ylabels = df['label']
X_tfidf = vectorizer.fit_transform(Xfeatures)
```

✓ 9.7s

Picture 20: Tokenize input text with modification

II. Validating best hyper parameter

A screenshot of a Jupyter Notebook cell with a dark background. The code uses sklearn's GridSearchCV to tune a pipeline consisting of a TfidfVectorizer and an SGDClassifier. The output shows the best parameters found: {'clf__alpha': 0.0001, 'clf__loss': 'modified_huber', 'clf__penalty': 'l2'} and the best accuracy of approximately 0.9083.

```
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import SGDClassifier
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd
import os

df = pd.read_csv("../code/resources/newPreprocess.csv")
df = df.dropna(subset=['Final_Text'])

X = df['Final_Text']
y = df['label']
pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(
        ngram_range=(1, 2),
        sublinear_tf=True,
        max_df=0.9,
        min_df=5,
        max_features=50000
    )),
    ('clf', SGDClassifier(class_weight='balanced'))
])
param_grid = {
    'clf__alpha': [1e-4, 1e-3, 1e-2],
    'clf__penalty': ['l2', 'l1', 'elasticnet'],
    'clf__loss': ['hinge', 'log_loss', 'modified_huber']
}
grid_search = GridSearchCV(pipeline, param_grid, cv=5, n_jobs=-1, scoring='accuracy', verbose=2)
grid_search.fit(X, y)
print("Best Parameters:", grid_search.best_params_)
print("Best Accuracy:", grid_search.best_score_)

✓ 6m 28.6s

Fitting 5 folds for each of 27 candidates, totalling 135 fits
Best Parameters: {'clf__alpha': 0.0001, 'clf__loss': 'modified_huber', 'clf__penalty': 'l2'}
Best Accuracy: 0.908335533036583
```

Picture 21: Validating best hyper parameter

As shown in picture 21, we apply GridSearchCV library to tuning for the best hyper parameter for logistic regression for this dataset. The parameter allowed to test include alpha (1e-4, 1e-3 and 1e-2) with lower alpha means less regularization and more chance for overfitting, penalty (l2 for Ridge, l1 for Lasso and elasticnet for the mixed of l1 and l2), and loss (hinge, log_loss and modified_huber).

We test on a 5 fold cross validation test and work for the best accuracy that can be achieved. The result illustrates that the best accuracy can be achieved is 91% with alpha = 0.0001, loss function is modified huber and penalty used is l2 (Euclidean).

As shown in picture 3, the Emotion dataset currently used is extremely unbalanced since classes with label 0 and 1 outweigh other classes, especially class 5 which negatively impact our model since it might ignore small classes and only predict the one with the major of instances. Therefore, we add 'balanced' for class_weight hyper parameter to increase the weight of small class (in this case, class 5) to penalize more when miss-classified small class.

III. Implementation

```

code > Data_analysis_and_preprocess.ipynb > import pandas as pd
Generate + Code + Markdown | Run All | Restart ... base (Python 3.11.7)

df = pd.read_csv("../code/resources/newPreprocess.csv")
df = df.dropna(subset=['Final_Text'])

vectorizer = TfidfVectorizer(
    ngram_range=(1, 2),
    sublinear_tf=True,
    max_df=0.9,
    min_df=5,
    max_features=50000,
)
Xfeatures = df['Final_Text']
ylabels = df['label']
X_tfidf = vectorizer.fit_transform(Xfeatures)
sgd_classifier = SGDClassifier(
    loss='modified_huber',
    penalty='l2',
    class_weight='balanced',
    alpha=0.0001,
    max_iter=1000,
    tol=1e-3
)

y_pred = cross_val_predict(sgd_classifier, X_tfidf, ylabels, cv=10)
y_scores = cross_val_predict(sgd_classifier, X_tfidf, ylabels, cv=10, method='cm')
cm = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.unique(ylabels))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.unique(ylabels))
disp.plot(cmap='Blues')
print("\nClassification Report:")
print(classification_report(ylabels, y_pred))
print(f"Accuracy: {accuracy_score(ylabels, y_pred):.4f}")
print(f"Precision: {precision_score(ylabels, y_pred, average='weighted'):.4f}")
print(f"Recall: {recall_score(ylabels, y_pred, average='weighted'):.4f}")
print(f"F1 Score: {f1_score(ylabels, y_pred, average='weighted'):.4f}")

# Binarize labels for PR/ROC curve
classes = np.unique(ylabels)
y_bin = label_binarize(ylabels, classes=classes)
if y_bin.shape[1] == 1:
    y_bin = np.hstack((1 - y_bin, y_bin)) # for binary case
    classes = [f'not-({classes[0]})', classes[0]]

# Ensure y_scores shape matches
if y_scores.ndim == 1 or y_scores.shape[1] != y_bin.shape[1]:
    # Fit manually to get decision function outputs with shape [n_samples, n_classes]
    sgd_classifier.fit(X_tfidf, ylabels)
    y_scores = sgd_classifier.decision_function(X_tfidf)
    if y_scores.ndim == 1:
        y_scores = np.vstack([-y_scores, y_scores]).T # binary case

# Plot Precision-Recall curves
plt.figure(figsize=(10, 6))
for i, class_name in enumerate(classes):
    precision, recall, _ = precision_recall_curve(y_bin[:, i], y_scores[:, i])
    pr_auc = auc(recall, precision)
    plt.plot(recall, precision, label=f'({class_name}) (AUC={pr_auc:.2f})')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# Plot ROC curves
plt.figure(figsize=(10, 6))
for i, class_name in enumerate(classes):
    fpr, tpr, _ = roc_curve(y_bin[:, i], y_scores[:, i])
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, label=f'({class_name}) (AUC={roc_auc:.2f})')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()
plt.grid(True)
plt.show()

```

Picture 22: Modification of logistic regression

As illustrated in picture 22, we utilize the sklearn library of Python to build this algorithm. In this algorithm, we apply modified huber for Loss measurement which applies mean squared root for small errors and mean absolute error for large errors which leads to a smoothed, outlier-tolerant, probabilistic classifier, as shown in formula below.

$$Loss = \frac{1}{2}(y_{true} - y_{pred})^2 \text{ if } (y_{true} - y_{pred}) < \delta \text{ or}$$

$$Loss = \delta(|y_{true} - y_{pred}| - \frac{1}{2}\delta) \text{ if } (y_{true} - y_{pred}) > \delta$$

Moreover, as shown in picture 22, we applied L2 (Euclidean) regularization with formula :

$$Original\ loss + \lambda \sum_i w_i^2 \text{ where } w \text{ is the weight of the model and } \lambda \text{ is regularization strength}$$

controlled by alpha hyperparameter declared in the code function. By applying this algorithm,

our model prevents overfitting and shrinks the model weight towards 0. Hence, the model becomes simpler and generalizes better on unseen data.

CHAPTER 5 : MODEL EVALUATION

I. Used formula definition

Accuracy is a metric usually being used to measure how correct a model prediction is [25].

$$Accuracy = \frac{\text{Number of correct prediction}}{\text{Total number of prediction}}$$

Precision is a measurement for a model to evaluate its performance by calculating the percentage of correctness when the model predicts Positives as shown in the formula below [16].

$$Precision = \frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}}$$

Recall, on the other hand, measures the ratio of total real True cases over True cases labelled by model [16]. Mathematically,

$$Recall = \frac{\text{True Positive}}{\text{True Positive} + \text{Negative}}$$

F1 score is the harmonic mean between precision and recall. If the F1 score is high, the model precision and recall are also high. In contrast, if the value of precision and recall are unbalanced, F1 score will be negatively affected [16].

$$F1\ score = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Precision curve is usually being used to graphically represent the classifier's Precision and Recall at different classification thresholds. The x-axis of the PR curve represents Recall while the y-axis represents Precision [26]. It is especially powerful in evaluating imbalance datasets where some classes outweigh the others.

ROC (Receiver Operating Characteristic) curve is an illustration of the relation of True Positive Rate and False Positive Rate[27]. It is a powerful tool to evaluate the performance of different models in classification tasks.

$$\text{True Positive rate} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}}$$

$$\text{False Positive rate} = \frac{\text{False Positive}}{\text{False Positive} + \text{False Negative}}$$

Macro average calculates each class performance metric and takes the arithmetic mean across all classes and gives the equal weight to each class, regardless of their number of instances [18].

$$\text{Macro average} = \frac{\sum_{i=1}^{\text{Number of class}} \text{Metric of class}_i}{\text{Number of class}}$$

Weight average is a method to compute an average where some data points contribute more than the others, or in other word, unbalanced dataset[17].

$$\text{Weight average} = \frac{\sum (\text{Metric of class} \times \text{Number sample of that class})}{\text{Total samples}}$$

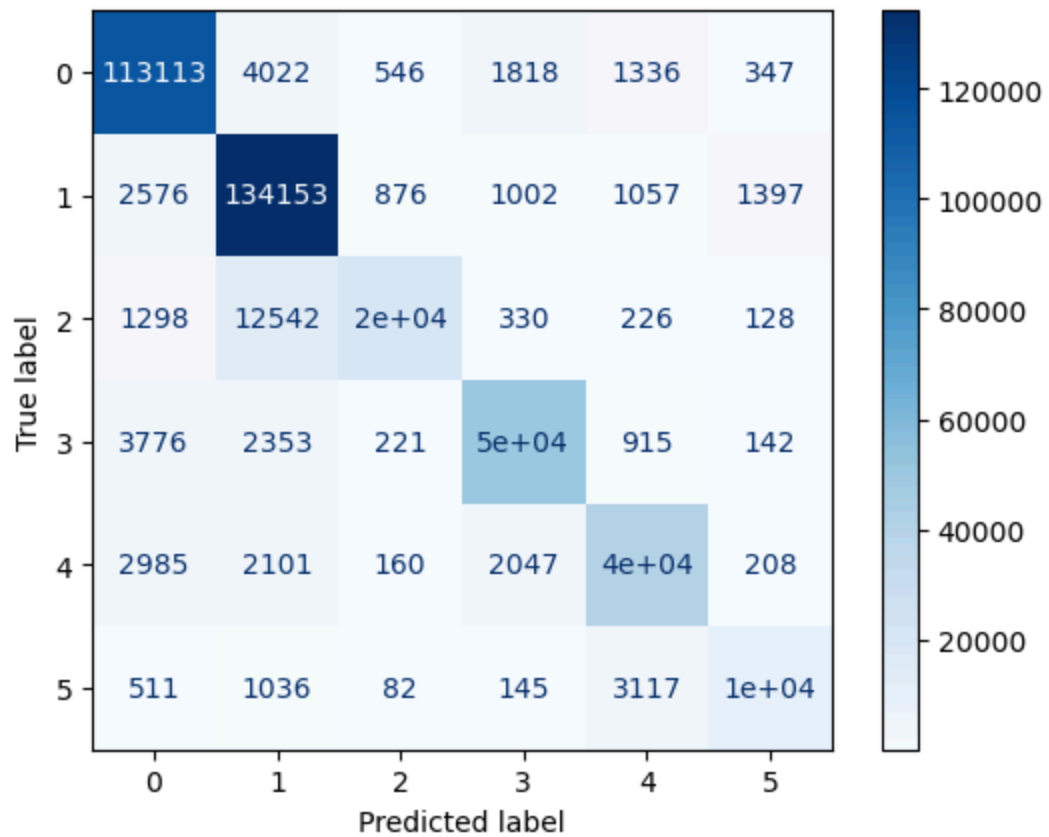
10-fold cross-validation is a technique to evaluate the performance of a machine learning model , especially when the data resource is limited and models' creators want to avoid overfitting. The implementation of this technique includes dividing the dataset into 10 equal parts, then training the model using 9 parts and testing on the remaining part and repeating this process for 10 times, each time using the different part for testing[19]. This technique allows users to have a more reliable estimate over the algorithm and model used since it supports testing over the whole dataset without overfitting.

II. Logistic Regression pre modification

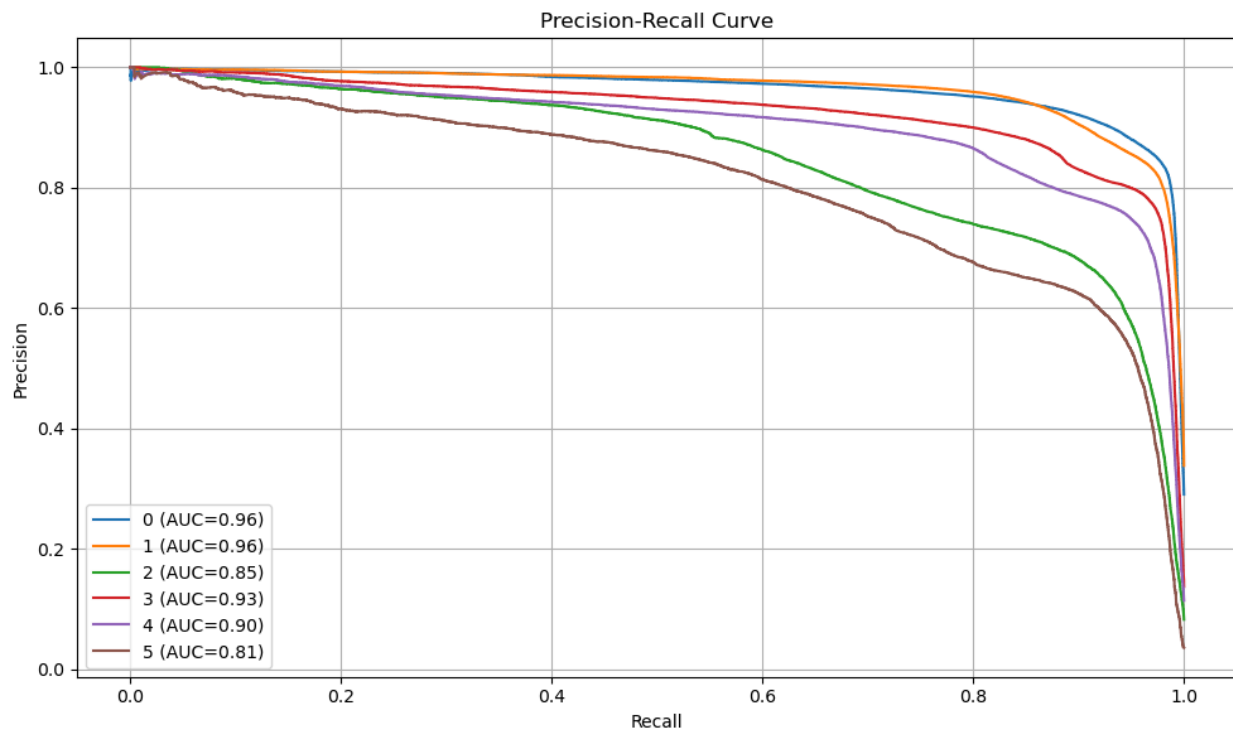
Classification Report:					
	precision	recall	f1-score	support	
0	0.91	0.93	0.92	121182	
1	0.86	0.95	0.90	141061	
2	0.91	0.58	0.71	34554	
3	0.90	0.87	0.89	57306	
4	0.86	0.84	0.85	47706	
5	0.82	0.67	0.74	14972	
accuracy			0.88	416781	
macro avg	0.88	0.81	0.83	416781	
weighted avg	0.88	0.88	0.88	416781	
Accuracy:	0.8817				
Precision:	0.8830				
Recall:	0.8817				
F1 Score:	0.8781				

Picture 23: Classification report of Stochastic descent optimizer for Logistic regression

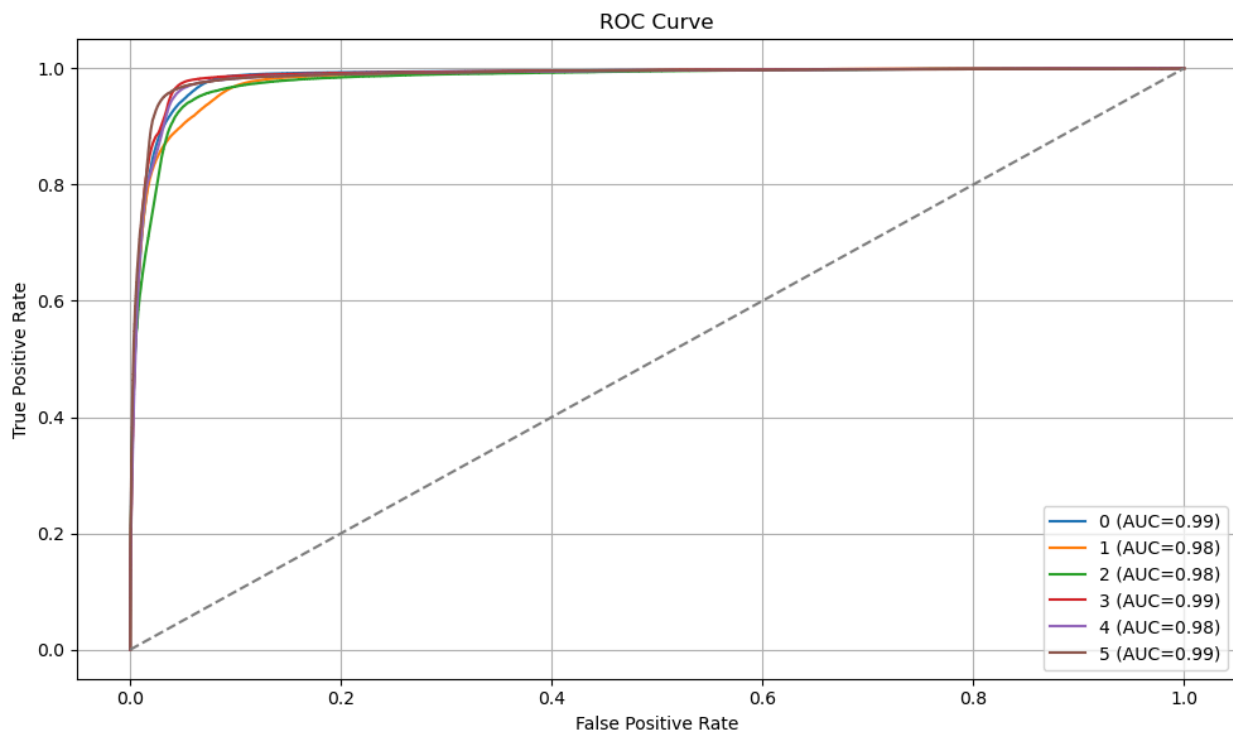
As shown in the picture 23, Logistic regression is shown to work well on this dataset. Overall, the precision is 0.8830 indicates that 88.3% is the probability that the model predicts the correct value, the recall measurement is 0.8817 shows that considering how many true examples are in each class, the model could correctly predict 88.17% of the true labels . The accuracy level of the model is higher than expectation with 88.17% illustrated that around 88% prediction of the model is corrected.



Picture 24: Heat map of Stochastic descent optimizer for Logistic regression



Picture 25: Precision curve of Stochastic descent optimizer for Logistic regression



Picture 26: ROC curve of Stochastic descent optimizer for Logistic regression

To be more specific for each class, the classification report shown in picture 23 and heat map shown in picture 24 clearly indicates that the model is very confident in predicting class 0 and class 1 with extremely high precision and recall statistics for both of those classes. However, in class 2 and 5, the corresponding recall are particularly high (0.91 and 0.82 respectively) while the precision values are much lower (0.58 and 0.67). It illustrates the fact that when the model predicts class 2, it is usually a right labeled prediction; however, with low recall it means they usually misclassified the class 2 with other labels, to be more specific, the model usually predict class 2 instances with class 0 and 1 (as shown in the heat map in picture 17). In class 5, the model also has trouble in identifying and correctly classifying class 5. The reason for this phenomenon is that the imbalance of the dataset as shown in picture 3 and the low support of class 2 and 5. As shown in picture 25 and picture 26, the precision curve and ROC curve illustrate that label 0, 1 and 3 precision curves are extraordinarily smooth with nearly approach 1 indicating that our model is extremely confident in defining these 3 classes. However, in class 2 and 5, the precision curves are fluctuating and reduce when recall value increases shows that the model is imbalanced for those 2 classes due to the shortage of instances as shown in picture 3. As I mentioned before, since the dataset is strongly unbalanced, it has negatively impacted the ROC curve since in underperformance class (2 and 5), the negative instances far more outweigh the positive label which leads to the high performance evaluation of class 2 and 5 in ROC curve.

One of the main advantages of Logistic regression for this dataset is the short time needed for training and testing even though the dataset is pretty large and 10 fold cross validation is time consuming. The time complexity of logistic regression, especially with stochastic descent regression is always an advantage when compared with other more complicated algorithms with Training time complexity equals $O(e \times n \times d)$ with e is number of epochs, n is training samples and d is number of features and Testing time complexity is $O(n \times d)$ with n is number of test samples and d is number of features.

III. Logistic regression post modification

```
...
Classification Report:

```

	precision	recall	f1-score	support
0	0.97	0.92	0.95	121182
1	0.96	0.90	0.93	141061
2	0.74	0.92	0.82	34554
3	0.91	0.92	0.91	57306
4	0.88	0.86	0.87	47706
5	0.68	0.98	0.80	14972
accuracy			0.91	416781
macro avg	0.86	0.92	0.88	416781
weighted avg	0.92	0.91	0.91	416781

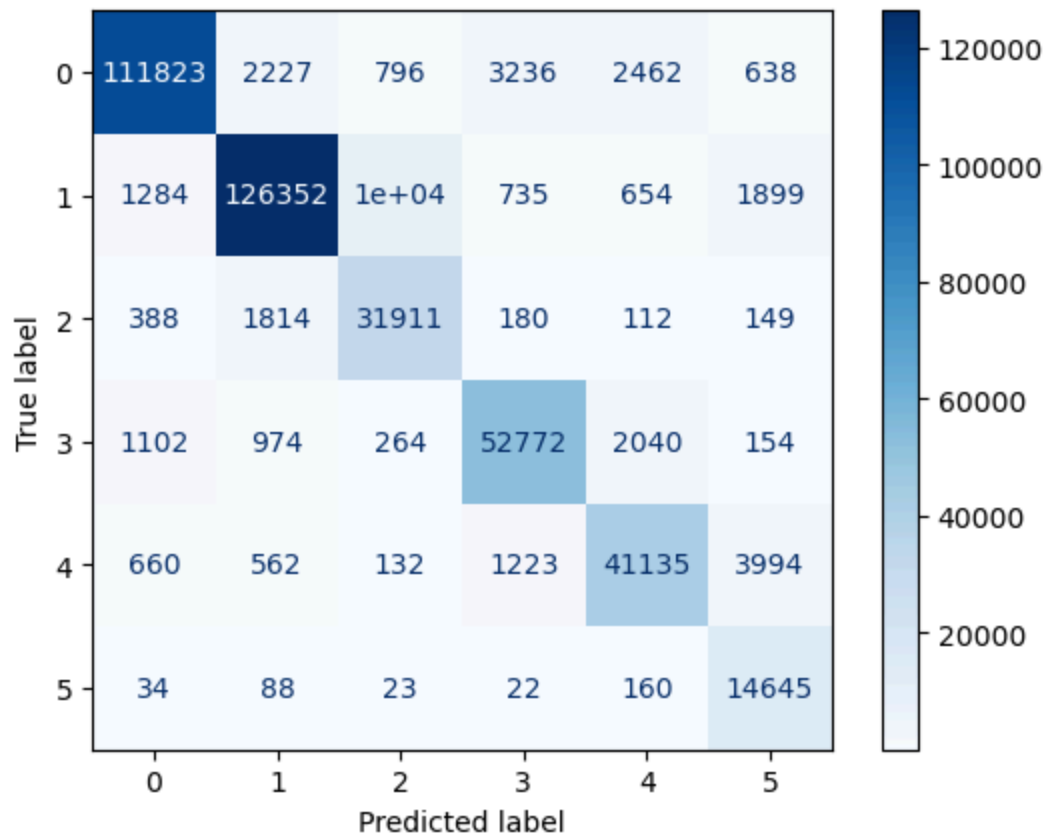
```

Accuracy: 0.9085
Precision: 0.9174
Recall: 0.9085
F1 Score: 0.9106

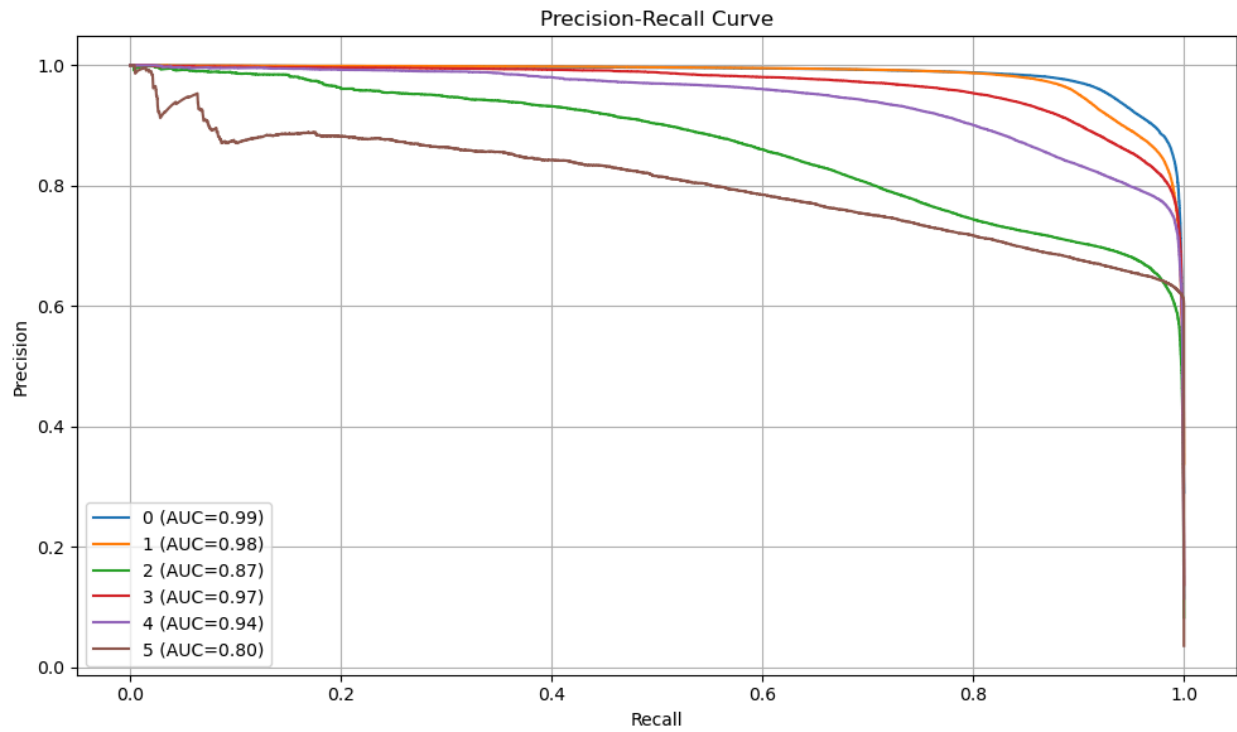
```

Picture 27: Classification report of modified Stochastic descent optimizer for Logistic regression

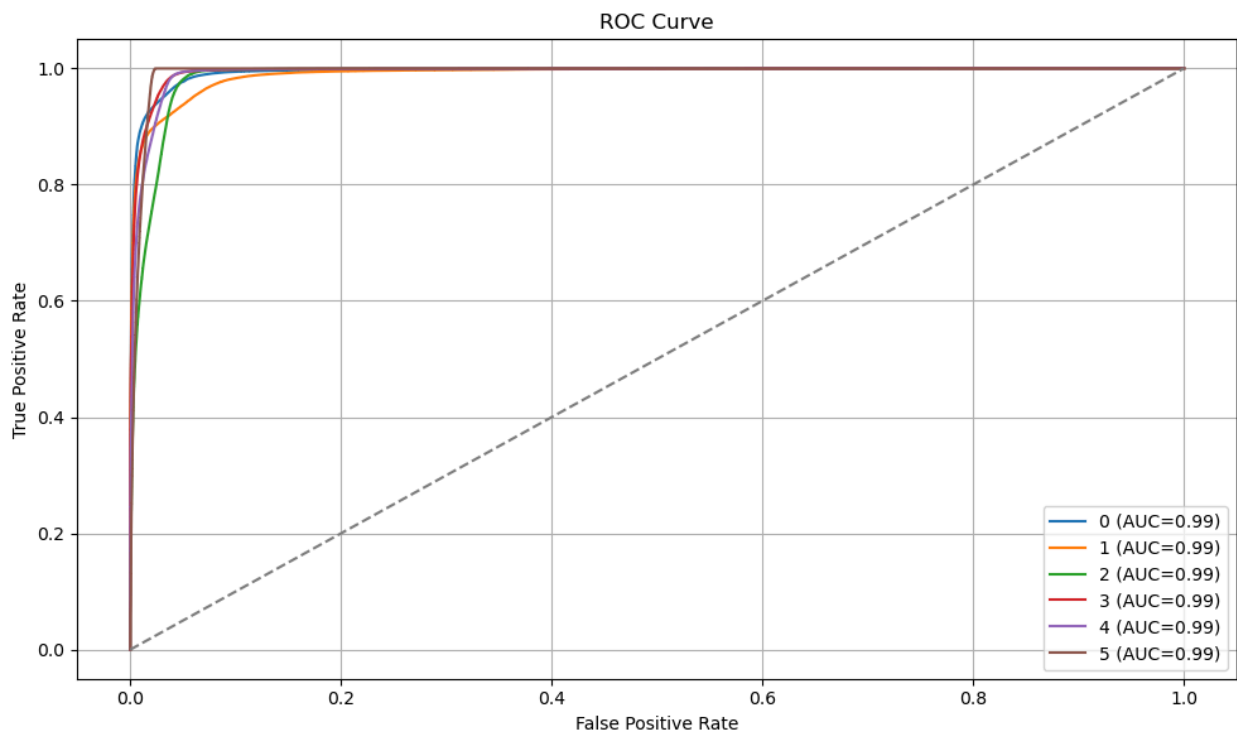
As shown in the picture 27, Logistic regression is shown to work well on this dataset. Overall, the precision is 0.9174 indicates that 91.7% is the probability that the model predicts the correct value, the recall measurement is 0.9085 shows that considering how many true examples are in each class, the model could correctly predict 90.85% of the true labels . The accuracy level of the model is higher than expectation with 90.85% illustrated that around 90% prediction of the model is corrected. Compared to the post modification, the model statistic has improved significantly, which is a good signal that our modification is on the right track.



Picture 28: Heat map of modified Stochastic descent optimizer for Logistic regression



Picture 29: Precision curve of modified Stochastic descent optimizer for Logistic regression



Picture 30: ROC curve of modified Stochastic descent optimizer for Logistic regression

To be more specific for each class, the classification report shown in picture 27 and heat map shown in picture 28 clearly indicates that the model is very confident in predicting class 0 and class 1 with extremely high precision and recall statistics for both of those classes. However, in class 2 and 5, the corresponding recall are particularly high (0.95 and 0.94 respectively) while the precision values are much lower (0.72 and 0.67). It illustrates the fact that while the model accurately classified all class 2 and 5 instances, it often wrongly predicted other classes as class 2 and 5 as well. The reason for this phenomenon is that the model is set up with `class_weight='balanced'`. However, this action is necessary in this case because as shown in picture 3, this dataset is strongly unbalanced with the majority of the instances are labeled with 0 or 1, only the very minimum percentage of the dataset are labeled with 2 or 5 which leads to the fact that the model easily ignores class 2 and 5 and tends to give class 0 and 1 more focuses. For that reason, `class_weight='balanced'` is the must in this case to encourage models to give more attention to the minority classes such as 2 and 5. However, compared to the post modification model version, the performance for class 2 and 5 has improved significantly with slightly lower precision but higher recall for both classes which indicates that the model catches more true positives but also a small number of true negatives for class 2 and 5. As shown in picture 29 and picture 30, the precision curve and ROC curve illustrate that label 0, 1 and 3 precision curves are extraordinarily smooth with nearly approach 1 indicating that our model is extremely confident in defining these 3 classes. However, in class 2 and 5, the precision curves are fluctuating and reduce when recall value increases shows that the model is imbalanced for those 2 classes due to the shortage of instances as shown in picture 3. As I mentioned before, since the dataset is strongly unbalanced, it has negatively impacted the ROC curve since in underperformance class (2 and 5), the negative instances far more outweigh the positive label which leads to the high performance evaluation of class 2 and 5 in ROC curve.

Compared to the pre-modification version, the statistical value has increased dramatically (accuracy, precision, recall and F1 score) which illustrates that the changes have a positive effect on the model performance. The performance for class 0, 1, and class 4 have improved significantly (with higher precision and recall specific for those classes); however, the class 2 and 5 have also been improved but not too clearly.

For the future modification to improve the mentioned drawbacks, I would like to add more data to diverse my datasets and improve the unbalanceness which will definitely improve the performance of both Logistic regression, especially on class 2 and class 5. Secondly, after gaining more dataset, improving tokenization is necessary to improve the overall performance as well. Lastly, for specific improvement of Logistic regression, I would like to apply other optimized methods such as Adam with higher impact on the weight convergence.

IV. Compare model with other models on the same field

With the same dataset, a number of algorithms have been introduced to solve the task of emotion classification such as multinomial Naive Bayes [29].

According to the report [29], the Gaussian model is not suitable for this dataset with low measurement statistics. However, Multinomial Naive Bayes and Complement Naive Bayes yields more positive results. Therefore, Multinomial Naive Bayes and Complement Naive Bayes were tested further with different tokens and the results illustrates that both Multinomial Naive Bayes and Complement Naive Bayes show better result for 5000 features compared to 2000 features but more than 5000 features shows negligible difference.

ComplementNB / 8k features

	precision	recall	f1-score	
0	0.93	0.93	0.93	
1	0.92	0.91	0.92	
2	0.79	0.82	0.81	
3	0.89	0.89	0.89	
4	0.86	0.85	0.86	
5	0.78	0.76	0.77	
accuracy			0.89	
macro avg	0.86	0.86	0.86	
weighted avg	0.89	0.89	0.89	

Picture 31: Classification report of complementNB of 8000 features

MultinomialNB / 8k features

	precision	recall	f1-score
0	0.91	0.92	0.91
1	0.90	0.91	0.90
2	0.76	0.76	0.76
3	0.89	0.85	0.87
4	0.84	0.84	0.84
5	0.76	0.69	0.73
accuracy			0.88
macro avg	0.84	0.83	0.84
weighted avg	0.88	0.88	0.88

Picture 32: Classification report of multinomialNB of 8000 features

According to the classification report of multinomial NB and complement NB in picture 31 and 32, our model post modification yields better results in both macro average and weighted average with higher precision, recall and f1 score. It's due to the fact that we have different approaches in both pre-processing data and building models. As shown in picture 31, 32 and 27, our model shows more confidence in predicting class 0, 1 and 3 which are the majority in the datasets. For class 4, the difference is negligible. However, for the minority classes (2 and 5), our model yields better results on recall but the multinomial NB and complementNB shows the harmonic in precision and recall statistics.

Moreover, besides logistic regression, we also test on Light Gradient Boosting Machine is an open source high-performance framework developed by Microsoft that utilizes gradient boosting method which constructs a strong learner by sequentially adding weak learners in a gradient descent manner[8]. By using tree based learning algorithms that can grow efficiently by minimizing memory usage and optimizing training time, it is designed for efficiency, scalability and high performance, especially for large datasets like our Emotion dataset [20].

Light Gradient Boosting Machine is an advanced and optimized version of the traditional Gradient Boosting Machine (GBM) which is a boosting algorithm that combines multiple weak learners to create a strong predictive model. Unlike the other popular algorithms that also create decision trees such as Random Forest[22], GBM did not build trees in parallel and independently but rather built them sequentially with each tree being trained to fix the errors made by the entire model so far (such as gradients or residuals)[21]. At each step, GBM adds a new tree to the model to predict the negative gradient of the loss function. It is similar to the function of logistic gradient descent aforementioned in chapter 3.

GBM's initialize model formula:

$$\widehat{F}_0(x) = \underset{c}{\operatorname{argmin}} \sum_{i=1}^n L(y_i, c)$$

In each stage, negative gradient of loss function, weak learner and optimal steps are measured by following formulas.

$$\text{Negative gradient of loss function: } r_{im} = - \left[\frac{\partial L(y_i, \widehat{F}(x_i))}{\partial \widehat{F}(x_i)} \right]_{\widehat{F}(x) = \widehat{F}_{m-1}(x)}$$

Fit weak learner (decision tree) to the residuals: $h_m(x) \approx r_{im}$

$$\text{Optimal step size: } \gamma_m = \underset{\gamma}{\operatorname{argmin}} \sum_{i=1}^n L(y_i, \widehat{F}_{m-1}(x_i) + \gamma h_m(x_i))$$

After that, the model is updated by the formula: $\widehat{F}_m(x) = \widehat{F}_{m-1}(x) + \gamma_m h_m(x)$

The loss function L can be used in GBM is varied and each has its own advantage and disadvantage as shown in table 5.

Loss function	Formula
Mean squared error	$L(y, \hat{y}) = (y - \hat{y})^2$
Mean Absolute Error	$L(y, \hat{y}) = \frac{1}{n} \sum_{i=1}^n y_i - \hat{y}_i $
Quantile loss	$L(y, \hat{y}) = \max(q(y - \hat{y}), (q - 1)(y - \hat{y}))$

Table 5: Loss function can be applied to GBM

Light GBM applied the same idea with GBM with several modification

- Gradient-based One-side sampling supports the model to prioritize data instances with large gradients to improve efficiency rather than random subsampling like the way GBM method does. Hence, the algorithm now can focus on high-error samples.
- Exclusive feature bundling allows the algorithm to combine sparse features to reduce dimensionality without significant loss of information.
- Leaf-wise tree growth strategy gives light GBM opportunity to expand its tree by depth first search algorithm rather than breadth first search algorithm like the way traditional GBM does. As a result, it expands only the leaf that reduces loss the most rather than level by level which leads to faster convergence rate, lower training error and more efficient since it skips unnecessary splits.
- Histogram-based binning for split finding allows the new version of GBM to bins continuous features into discrete buckets. Therefore, it is not necessary to check every value but rather evaluate between bins with time complexity $O(\text{features} \times \text{bins})$ since in the traditional version, the time complexity is $O(\text{features} \times \text{data} \times \log(\text{data}))$. Hence, it reduces times needed for training and testing and lower memory usage with the approximate optimal.

```

import lightgbm as lgb
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score, cross_val_predict
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, precision_score, recall_score, f1_score, classification_report
import numpy as np

clf = lgb.LGBMClassifier(class_weight='balanced', n_estimators=200, n_jobs=-1)
y_pred = cross_val_predict(clf, X_tfidf, ylabels, cv=10)
cm = confusion_matrix(ylabels, y_pred, labels=np.unique(ylabels))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.unique(ylabels))
disp.plot(cmap='Blues')
print("\nClassification Report:")
print(classification_report(ylabels, y_pred))
precision = precision_score(ylabels, y_pred, average='weighted')
recall = recall_score(ylabels, y_pred, average='weighted')
f1 = f1_score(ylabels, y_pred, average='weighted')

print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1 Score: {f1:.4f}")

```

Picture 33: Light Gradient Boosting Machine implementation

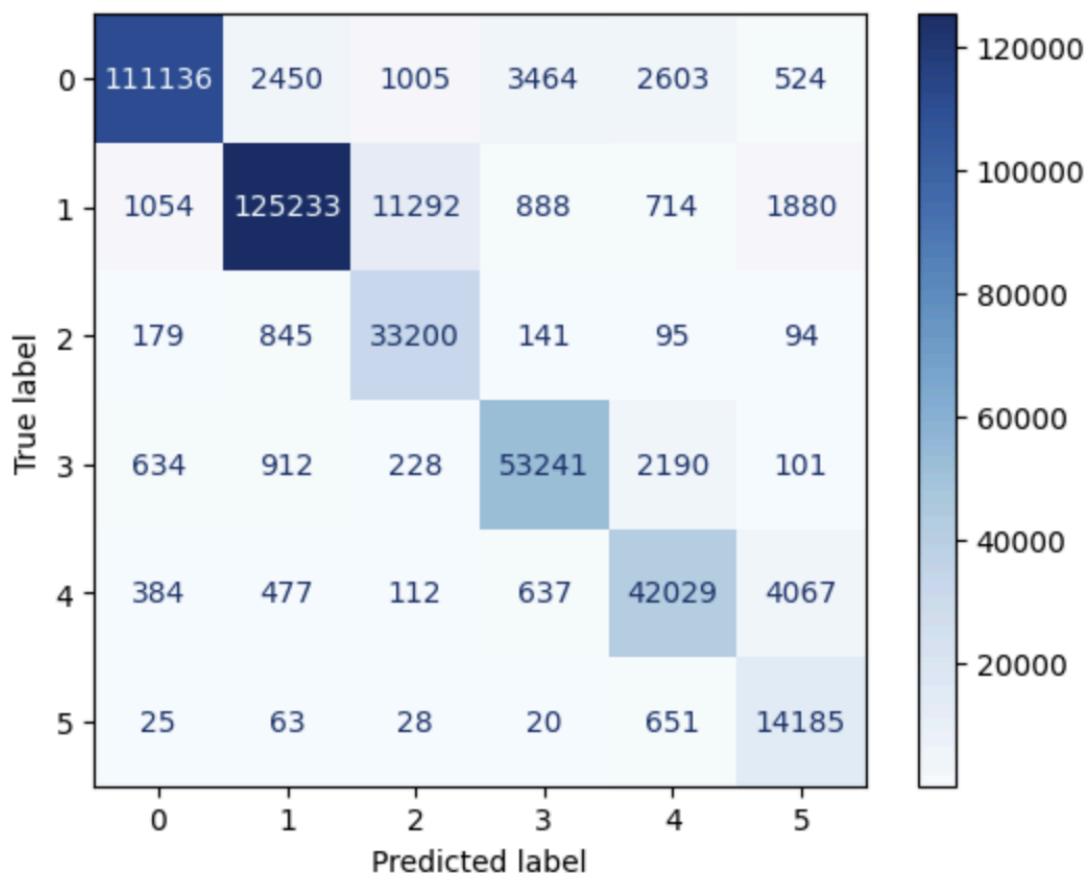
Classification Report:

	precision	recall	f1-score	support
0	0.98	0.92	0.95	121182
1	0.96	0.89	0.92	141061
2	0.72	0.96	0.83	34554
3	0.91	0.93	0.92	57306
4	0.87	0.88	0.88	47706
5	0.68	0.95	0.79	14972
accuracy			0.91	416781
macro avg	0.85	0.92	0.88	416781
weighted avg	0.92	0.91	0.91	416781

Precision: 0.9205
 Recall: 0.9094
 F1 Score: 0.9119

Picture 34: Classification report of Light Gradient Boosting Machine

As shown in picture 34, the performance of Light Gradient Boosting Machine has slightly increased compared to logistic regression aforementioned but negligible. Overall, the precision is 92.05% (compared to 91.74% of logistic regression) and the recall is 90.94% (compared to 90.85% of logistic regression) which proved that precision and recall are harmonic which leads to the high value of F1 score. With the high percentage of precision and recall, hence there is a very low possibility for false positives and false negatives. However, the price for this slight update is costly since compared with logistic regression, time spent for training and testing is far higher than with higher time complexity for training is $O(\text{features} \times \text{bins})$.



Picture 35: Heat map of Light Gradient Boosting Machine

As illustrated in picture 34 and picture 35, for class 0 and 1, which are the majority of the dataset, the precision and recall of those classes are particularly high which proves that the model possibility to give a true prediction for those classes is also high and slightly over performed

logistic regression. However, in minority classes, such as class 2 and 5, logistic regression models over performed the light GBM.

In conclusion, Logistic regression outweighs both Naive Bayes and light GBM on the time spent for both training and testing. Moreover, due to the simplicity of the model, it can highly avoid overfitting. Compared about the performance and confidence, Logistic regression and light GBM out perform Naive Bayes and the two show nearly similar statistics (in precision and recall). Logistic regression slightly under performs in the majority class compared to light GBM; however, in the minority class, logistic regression yields better results.

CHAPTER 6

CONCLUSION

I. Conclusion

Over the course of two and a half months, our team focused on the development of Sentimental Analysis Chatbot, and finished with a delivery of the Minimum Viable Product (MVP). The project was undertaken by a team of three members, leveraging the knowledge and skills acquired from the Data mining course led by the instructor Dr. Nguyen Thi Thanh Sang. The project completed at around 90% compared to the project plan, as discussed in details in Chapter 5: Model Evaluation.

Our team employed a concise development procedure, encompassing the following stages: Preprocessing data, Analyzing data, Implementing and optimizing two data mining algorithms, and Evaluating their performance.

The team was organized into modular units, each focusing on a specific aspect of the project led by a team leader who communicates directly with the project manager. Overall, all members worked collaboratively and contributed with great efforts to the project.

II. Future Prospects

The completion of the Minimum Viable Product still leaves various future features that can be implemented in the future. The list includes:

- + Web based for the chat bot
- + Optimizing the tokenizing data process to enhance accuracy
- + Applying higher complexity algorithm such as XGboost to maximize accuracy
- + Training and testing on different datasets to ensure the performance of models for real data.

REFERENCES

1. Emotion Dataset from Kaggle
2. <https://airbyte.com/data-engineering-resources/text-analysis-in-python>
3. <https://jcharis.github.io/neattext/>
4. <https://docs.python.org/3/library/re.html>
5. <https://pypi.org/project/pycontractions/>
6. <https://www.ibm.com/think/topics/knn>
7. <https://scikit-learn.org/stable/>
8. <https://www.geeksforgeeks.org/lightgbm-light-gradient-boosting-machine/>
9. <https://machinelearningcoban.com/2017/01/27/logisticregression/>
10. <https://arxiv.org/abs/1412.6980>
11. <https://aws.amazon.com/what-is/sentiment-analysis/>
12. <https://www.geeksforgeeks.org/what-is-sentiment-analysis/>
13. <https://paperswithcode.com/dataset/sst>
14. <https://www.kaggle.com/datasets/saurav9786/amazon-product-reviews>
15. <https://alternacx.com/blog/social-media-sentiment-analysis-the-rise-of-ai/>
16. <https://www.britannica.com/science/precision-and-recall>
17. <https://www.ncl.ac.uk/webtemplate/ask-assets/external/maths-resources/statistics/descriptive-statistics/weighted-averages.html#:~:text=Definition,the%20same%20as%20the%20mean.>
18. <https://www.evidentlyai.com/classification-metrics/multi-class-metrics#:~:text=Macro%20Daveraging%20calculates%20each%20class's,of%20the%20number%20of%20instances.>
19. <https://machinelearningmastery.com/k-fold-cross-validation/>
20. <https://lightgbm.readthedocs.io/en/stable/>
21. <https://www.geeksforgeeks.org/ml-gradient-boosting/>
22. <https://www.baeldung.com/cs/gradient-boosting-trees-vs-random-forests>
23. <https://pypi.org/project/lightgbm/>
24. <https://github.com/JDACS4C-IMPROVE/LGBM>
25. <https://viblo.asia/p/danh-gia-model-trong-machine-learning-RnB5pAq7KPG>

26. <https://www.geeksforgeeks.org/precision-recall-curve-ml/>
27. <https://statio.vn/blog/roc-curve-la-gi-giai-thich-thuat-ngu-cach-oc-roc-curve-va-y-nghia-tong-phan-tich-mo-hinh>
28. <https://www.kaggle.com/code/alexbostock/naive-bayes>
29. <https://www.kaggle.com/code/deep Patel8683/emotion-analysis>